

Piecewise polynomial interpolation

- (a) Waarom passen we in situaties PPI toe in de plaats van normal polynomial interpolation?
- (b) Hoeveel vrij te kiezen parameters zijn er voor een cubic spline?
- (c) Welke voorwaarden zorgen voor het aantal vastgelegde parameters en het aantal vrije parameters?
- (d) Op welke manieren kunnen we de twee vrije parameters van de cubic spline invullen?

Antwoord.

- (a) We passen piecewise polynomial interpolation (PPI) toe omdat een enkel globaal interpolatiepolynoom van hoge graad numeriek problematisch is. Dergelijke polynomen vertonen vaak sterke oscillaties, vooral nabij de randen van het interval (Runge-fenomeen), en het bijhorende interpolatieprobleem is slecht geconditioneerd. Daarnaast is een globaal polynoom niet lokaal: een kleine wijziging in één datapunt beïnvloedt de interpolant over het volledige domein. PPI vermijdt deze problemen door het interval op te splitsen en op elk subinterval een polynoom van lage graad te gebruiken, waardoor de interpolant lokaal, stabiel en numeriek robuust blijft.
- (b) Voor een cubic spline blijven er precies 2 vrij te kiezen parameters over.
- (c) Beschouw n knooppunten $x_0, \dots, x_{n-1}$. Er zijn $n - 1$ subintervallen en op elk subinterval staat een kubisch polynoom met 4 coëfficiënten, wat in totaal $4(n - 1)$ onbekenden oplevert.

Deze onbekenden worden vastgelegd door:

- Interpolatievoorwaarden: de spline moet door elk datapunt gaan, wat $2(n - 1)$ voorwaarden oplevert (twee per subinterval).
- Continuïteit van de eerste afgeleide: in elk intern knooppunt $x_1, \dots, x_{n-2}$ moet S' continu zijn, wat $n - 2$ voorwaarden geeft.
- Continuïteit van de tweede afgeleide: in elk intern knooppunt $x_1, \dots, x_{n-2}$ moet S'' continu zijn, wat nog eens $n - 2$ voorwaarden geeft.

In totaal zijn er dus $2(n - 1) + (n - 2) + (n - 2) = 4n - 6$ voorwaarden. Aangezien er $4(n - 1) = 4n - 4$ onbekenden zijn, blijven er 2 vrije parameters over die via randvoorwaarden moeten worden ingevuld.

- (d) De twee vrije parameters van een cubic spline worden bepaald door het kiezen van twee randvoorwaarden. Naast de meest gebruikte keuzes bestaan er meerdere mogelijkheden, waaronder:
 - **Clamped spline (complete spline):** de eerste afgeleide in beide eindpunten wordt opgelegd, dus $S'(x_0)$ en $S'(x_{n-1})$ zijn gegeven.
 - **Natural spline:** de tweede afgeleide in beide eindpunten wordt nul genomen, dus $S''(x_0) = 0$ en $S''(x_{n-1}) = 0$.
 - **Not-a-knot spline:** extra gladheid wordt afgedwongen door continuïteit van de derde afgeleide in de eerste en laatste interne knoop, wat betekent dat de eerste twee en laatste twee segmenten samen één kubisch polynoom vormen.
 - **Periodic spline:** de spline en haar eerste en tweede afgeleide zijn periodiek, zodat $S(x_0) = S(x_{n-1})$, $S'(x_0) = S'(x_{n-1})$ en $S''(x_0) = S''(x_{n-1})$ gelden.
 - **Mixed boundary conditions:** een combinatie waarbij bijvoorbeeld de eerste afgeleide aan één uiteinde en de tweede afgeleide aan het andere uiteinde wordt opgelegd.
 - **Symmetry conditions:** randvoorwaarden die voortkomen uit bekende symmetrie van het probleem, bijvoorbeeld een nul eerste afgeleide aan een symmetrie-as.

Al deze keuzes vullen de twee vrijheidsgraden in en leiden tot een unieke cubic spline, maar ze modelleren verschillende aannames over het gedrag van de interpolant aan de randen.

Fast Fourier Transform

- (a) Aan welke (3) voorwaarden moet een time series voldoen om er FFT op toe te passen?
- (b) Moet er een geheel aantal periodes in de dataset zitten, en wat gebeurt er als dat niet zo is?
- (c) Op welke andere soorten datasets dan time series kunnen we FFT toepassen?
- (d) Waarom doen we soms aan zero padding?

Antwoord.

- (a) Om de FFT zinvol toe te passen op een time series moeten drie kernvoorwaarden vervuld zijn. Ten eerste moet de data gelijkmatig gesampled zijn, omdat de FFT uitgaat van een constante tijdstap tussen opeenvolgende meetpunten. Ten tweede wordt de dataset door de FFT impliciet geïnterpreteerd als één periode van een periodiek signaal, wat betekent dat begin- en eindpunt conceptueel aan elkaar worden geplakt. Ten derde veronderstelt de klassieke radix-2 FFT dat het aantal datapunten van de vorm $N = 2^p$ is, al bestaan er ook FFT-algoritmen die met andere lengtes kunnen omgaan, zij het soms minder efficiënt.
- (b) Er hoeft geen geheel aantal periodes in de dataset te zitten, maar als dat niet het geval is ontstaat er een probleem aan de randen. Wanneer het signaal niet netjes aansluit bij periodieke herhaling, verschijnen er discontinuïteiten tussen het einde en het begin van de dataset. Deze kunstmatige sprongen introduceren extra hoge frequenties, wat leidt tot spectral leakage: energie die in werkelijkheid geconcentreerd is op één frequentie wordt uitgesmeerd over meerdere frequentiecomponenten in het spectrum.
- (c) FFT kan worden toegepast op elke dataset die op een uniform rooster ligt. Naast tijdreeksen zijn dat bijvoorbeeld ruimtelijke meetreeksen, waarbij de frequentie-as overeenkomt met golftallen. Ook tweedimensionale en driedimensionale datasets, zoals beelden of volumetrische data, kunnen met een multidimensionale FFT worden geanalyseerd. In het algemeen is de enige vereiste dat de data regelmatig gesampled is in de relevante dimensies.
- (d) Zero padding wordt om meerdere redenen toegepast. Door nullen aan de dataset toe te voegen, wordt het spectrum geëvalueerd op een fijner frequentierooster, waardoor pieken visueel scherper gelokaliseerd worden, zonder dat de echte spectrale resolutie toeneemt. Daarnaast is zero padding essentieel bij FFT-gebaseerde convoluties om circulaire convolutie te vermijden en een lineaire convolutie te bekomen. Ten slotte wordt zero padding vaak gebruikt om de lengte van de data aan te passen aan een FFT-lengte die computationeel efficiënter is.

Code

- (a) Bekijk de twee algoritmen. Wat berekenen deze?
- (b) Bespreek de twee algoritmen op vlak van efficiëntie en nauwkeurigheid. Welke van de twee is het nauwkeurigst?
- (c) Hoe kan je de nauwkeurigheid van het nauwkeurigste algoritme nog verbeteren?

Antwoord.

- (a) Beide algoritmen berekenen de standaardafwijking van de waarden in de lijst `lis`, met normalisatie door N . Concreet wordt eerst een schatting van het gemiddelde gevormd en vervolgens de wortel genomen van de variantie. Het eerste algoritme doet dit expliciet via de gemiddelde kwadratische afwijking ten opzichte van het gemiddelde. Het tweede algoritme gebruikt de identiteit $\text{Var}(X) = E[X^2] - E[X]^2$ en neemt daarna de vierkantswortel.
- (b) Beide algoritmen hebben een computationele complexiteit van orde $O(N)$. Het tweede algoritme is iets efficiënter, omdat het slechts één lus over de data nodig heeft, terwijl het eerste algoritme twee lussen gebruikt. Qua nauwkeurigheid is het eerste algoritme echter superieur. In het tweede algoritme wordt $E[X^2] - E[X]^2$ berekend als verschil van twee getallen die numeriek dicht bij elkaar kunnen liggen wanneer de variantie klein is ten opzichte van het gemiddelde. Dit leidt tot sterke numerieke cancelatie en verlies aan significante cijfers. Het eerste algoritme vermijdt deze subtractie van bijna gelijke grootheden en is daarom numeriek stabiel.
- (c) De nauwkeurigheid van het eerste algoritme kan verder verbeterd worden door stabielere sommatie toe te passen. In plaats van een gewone optelling kan men gecompenseerde sommatie gebruiken, zoals de Kahan- of Neumaier-sommatie, zowel bij het berekenen van het gemiddelde als bij het optellen van de kwadratische afwijkingen. Hierdoor wordt de accumulatie van afrondingsfouten verminderd, vooral voor lange lijsten of data met sterk uiteenlopende groottes.

```
def func1(lis):
    N = len(lis)
    var1 = 0
    for num in lis:
        var1 += num
    var1 /= N
    var2 = 0
    for num in lis:
        var2 += (num - var1)**2

    var2 /= N
    return var2**(0.5)

def func2(lis):
    N = len(lis)
    var1 = 0
    var2 = 0

    for num in lis:
        var1 += num
        var2 += num**2

    var1 /= N
    var2 /= N

    var3 = var2 - var1**2

    return var3**0.5
```

Optimalisatie versus *root finding*

- (a) Beschouw de ééndimensionale methodes in het kader van optimalisatie en *root finding* vraagstukken. Welke methode is equivalent aan de *bisection* methode?
- (b) Welke methode is equivalent aan de *secant* methode?
- (c) Hoe verschilt de Newton methode van de *secant* methode?
- (d) Waarom kunnen we geen lineaire interpolatie uitvoeren om een minimum te vinden?
- (e) Bespreek het verschil tussen inverse (kwadratische) interpolatie bij nulpunten en reguliere kwadratische interpolatie bij minima. Illustreer beide methodes aan de hand van een schets.

Antwoord.

- (a) In ééndimensionale optimalisatie is de methode die equivalent is aan de *bisection* methode de *golden section search*. Beide methodes zijn bracketende methodes: ze vertrekken van een interval waarvan men weet dat de oplossing erin ligt en verkleinen dit interval systematisch zonder gebruik te maken van afgeleiden. Bij *bisection* wordt het teken van f gebruikt om een nulpunt te lokaliseren, terwijl bij *golden section search* enkel functiewaarden worden vergeleken om een minimum te bracketen.
- (b) De methode die equivalent is aan de *secant* methode in optimalisatie is de methode van *parabolische interpolatie*, meer specifiek inverse kwadratische interpolatie voor minima. Net zoals de *secant* methode geen afgeleiden gebruikt maar een lokale lineaire benadering van f opbouwt, gebruikt *parabolische interpolatie* enkel functiewaarden om een lokaal kwadratisch model van f te construeren en daaruit een nieuwe schatting voor het minimum te halen.
- (c) De Newton methode gebruikt expliciet afgeleide-informatie, namelijk zowel de eerste als de tweede afgeleide van de functie, om een lokaal kwadratisch model van de functie op te bouwen. De *secant* methode daarentegen vervangt afgeleiden door eindige verschillen en gebruikt enkel functiewaarden. Hierdoor convergeert Newton typisch kwadratisch nabij de oplossing, terwijl de *secant* methode slechts superlineaire convergentie heeft. Het nadeel van Newton is dat ze afgeleiden vereist en gevoelig is voor een slechte beginwaarde, terwijl de *secant* methode robuuster maar trager is.
- (d) Lineaire interpolatie kan geen minimum bepalen omdat een lineaire functie geen intern extremum heeft. Het minimum van een rechte ligt altijd in een van de eindpunten van het interval. Bij optimalisatie is juist een intern punt nodig waar de afgeleide nul wordt. Daarom is minstens een kwadratische benadering nodig om een lokaal minimum te kunnen modelleren.
- (e) Bij *inverse kwadratische interpolatie* voor nulpunten wordt niet $f(x)$ als functie van x geïnterpoleerd, maar wordt de inverse relatie $x = f^{-1}(y)$ benaderd met een kwadratische veelterm op basis van de punten $(f(x_i), x_i)$. De volgende iteratie wordt bekomen door deze interpolant te evalueren in $y = 0$, wat rechtstreeks een nieuwe schatting voor het nulpunt oplevert.

Bij reguliere kwadratische interpolatie voor minima wordt daarentegen een kwadratische veelterm $p(x)$ gefit door drie punten $(x_i, f(x_i))$. Het volgende iteratiepunt is het minimum van deze parabool, dus het punt waar $p'(x) = 0$. In beide gevallen wordt een lokaal kwadratisch model gebruikt, maar bij nulpunten wordt het model geïnverteerd, terwijl bij minima rechtstreeks het functieverloop wordt benaderd.

Interpolatie

- (a) Rangschik de volgende methodes in termen van computationele kost.
- (a) *monomial* basis
 - (b) Lagrange basis
 - (c) Newton basis
- (d) Hoe verschillen de geassocieerde coëfficiëntenmatrices in termen van conditionering?
- (e) Bij welke interpolant is het het eenvoudigst om een veelterm te evalueren in een zeker punt?
- (f) Breng de volgende veelterm in een geschikte vorm om Horner's methode op toe te passen.
- $$p(t) = 7t^3 - 3t^2 + 5t - 2$$

Antwoord.

- (a) In termen van computationele kost kunnen de methodes als volgt gerangschikt worden, van minst naar meest efficiënt:
- **Monomial basis:** deze vereist het oplossen van een volledig Vandermonde-stelsel, wat duur is en numeriek problematisch voor grotere n .
 - **Lagrange basis:** expliciete oplossing van een stelsel is niet nodig, maar evaluatie en herberekening bij nieuwe punten is relatief duur.
 - **Newton basis:** deze laat een recursieve opbouw toe via gedeelde verschillen en is computationeel het efficiëntst, zeker wanneer punten worden toegevoegd.
- (b) De conditionering van de coëfficiëntenmatrices verschilt sterk per basis. Bij de monomial basis ontstaat een Vandermonde-matrix, die zeer slecht geconditioneerd is, vooral voor grote n of niet-goed gekozen knooppunten. De Lagrange basis vermijdt expliciet een matrixformulering, maar is conceptueel equivalent en erft dezelfde conditioneringsproblemen. De Newton basis leidt tot een lagere-triangelulaire structuur in termen van gedeelde verschillen en is aanzienlijk beter geconditioneerd, omdat de interpolatie stapsgewijs wordt opgebouwd.
- (c) De Newton-interpolant is het eenvoudigst te evalueren in een gegeven punt. Door de geneste structuur van de Newton-vorm kan de veelterm efficiënt geëvalueerd worden met een Horner-achtige evaluatie, wat zowel snel als numeriek stabiel is.
- (d) Om Horner's methode toe te passen, herschrijven we de veelterm in geneste vorm. Voor $p(t) = 7t^3 - 3t^2 + 5t - 2$ krijgen we:

$$p(t) = ((7t - 3)t + 5)t - 2.$$

In deze vorm kan de veelterm met een minimaal aantal vermenigvuldigingen en optellingen geëvalueerd worden.

Kwadratuur

- (a) Wat houdt numerieke kwadratuur in? Bespreek Newton–Cotes, Clenshaw–Curtis en Gaussische kwadratuur.
- (b) Wat zijn de eerste drie Newton–Cotes regels? Illustreer ze aan de hand van een schets.
- (c) Stel dat we een hogere nauwkeurigheid wensen. Zou het een goed idee zijn om een derde orde veelterm te gebruiken in plaats van Simpsons regel? Verklaar waarom wel of niet, en illustreer dit aan de hand van je schets bij de vorige deelvraag.

Antwoord.

- (a) Numerieke kwadratuur heeft als doel een integraal $\int_a^b f(x) dx$ te benaderen door een gewogen som van functiewaarden, $\sum_{i=0}^n w_i f(x_i)$, waarbij de keuze van knooppunten x_i en gewichten w_i de nauwkeurigheid bepaalt.

Bij **Newton–Cotes** worden de knooppunten gelijkmatig gekozen op $[a, b]$ en wordt f geïnterpoleerd door een veelterm, die vervolgens exact geïntegreerd wordt. Deze regels zijn eenvoudig maar kunnen bij hogere orde numeriek onstabiel worden.

Clenshaw–Curtis gebruikt eveneens polynomiale interpolatie, maar met Chebyshev-knooppunten. Hierdoor wordt de oscillatie van de interpolerende veelterm beperkt en is de methode stabiel dan klassieke hoge-orde Newton–Cotes, terwijl ze efficiënt kan worden geïmplementeerd via FFT-technieken.

Gaussische kwadratuur kiest zowel knooppunten als gewichten optimaal zodat de methode exact is voor polynomen tot zo hoog mogelijke graad. Voor n knooppunten is de regel exact tot graad $2n - 1$. Dit maakt Gaussische kwadratuur zeer efficiënt, maar de knooppunten liggen niet uniform en zijn moeilijker te berekenen.

- (b) De eerste drie Newton–Cotes regels zijn:
 - de **rechthoekregel**, waarbij f wordt benaderd door een constante op het interval;
 - de **trapeziumregel**, waarbij f lineair wordt geïnterpoleerd tussen de eindpunten;
 - de **Simpsonregel**, waarbij f wordt benaderd door een kwadratische veelterm die door de twee eindpunten en het midden van het interval gaat.

In elk geval wordt de integraal benaderd door de oppervlakte onder respectievelijk een rechthoek, een trapezium en een parabool, wat men schematisch kan voorstellen door de grafiek van f en de corresponderende interpolerende veelterm op $[a, b]$ te tekenen.

- (c) Het gebruik van een derde-orde interpolerende veelterm in plaats van de Simpsonregel is in het algemeen geen goed idee. Hoewel een hogere graad intuïtief meer nauwkeurigheid suggereert, leidt een Newton–Cotes regel van hogere orde tot grotere oscillaties van de interpolerende veelterm tussen de knooppunten. Dit kan de fout zelfs vergroten en de methode numeriek instabiel maken. Simpson is exact voor polynomen tot graad drie en vormt een goed compromis tussen nauwkeurigheid en stabiliteit. Hogere nauwkeurigheid wordt beter bereikt door het interval op te splitsen en Simpson meervoudig toe te passen, of door over te stappen op Clenshaw–Curtis of Gaussische kwadratuur in plaats van de polynomiale graad verder te verhogen.

Explain each method and how it improves from previous: direct search, steepest descent, Newton, secant updating, conjugate gradient. Extra vragen: geef geometrische interpretaties en bespreek hoe we globale minima kunnen vinden.

Antwoord.

Direct search. Direct search methodes zoeken een minimum zonder gebruik te maken van afgeleiden en baseren zich enkel op functiewaarden. Typische voorbeelden zijn patroonzoekmethodes en simplex-achtige methodes. De verbetering ten opzichte van volledig willekeurig zoeken is dat het zoekgebied systematisch wordt verkleind op basis van functiewaarden. Geometrisch kan men dit zien als het verplaatsen en verkleinen van een meetkundige structuur in de oplossingsruimte totdat geen verdere daling meer mogelijk is. Deze methodes zijn robuust maar traag en schalen slecht met de dimensie.

Steepest descent. Steepest descent gebruikt de gradiënt van de functie en beweegt in de richting van de grootste lokale afname, dus in de richting $-\nabla f(x)$. Ten opzichte van direct search gebruikt deze methode lokale afgeleide-informatie en convergeert daardoor sneller. Geometrisch volgt de iteratie de richting loodrecht op de niveaulijnen van f . Het nadeel is dat de methode kan zigzaggen in langgerekte valleien, wat leidt tot trage convergentie.

Newton methode. De Newton methode verbetert steepest descent door ook tweede-orde informatie te gebruiken via de Hessiaan. In plaats van een lineair lokaal model wordt een kwadratisch model van de functie opgebouwd. Geometrisch komt dit overeen met het benaderen van f door een paraboloïde en rechtstreeks naar het minimum daarvan springen. Dit leidt tot kwadratische convergentie nabij het minimum, maar vereist het berekenen en oplossen van een lineair stelsel met de Hessiaan en kan falen als de Hessiaan niet positief definit is.

Secant updating. Secant updating methodes, zoals quasi-Newton methodes, vermijden de expliciete berekening van de Hessiaan door deze iteratief te benaderen met informatie uit opeenvolgende iteraties. Ze verbeteren Newton doordat ze goedkoper zijn per iteratie, terwijl ze toch superlineaire convergentie behouden. Geometrisch wordt de kromming van de functie gaandeweg geleerd in plaats van exact berekend.

Conjugate gradient. De conjugate gradient methode is ontworpen voor kwadratische functies en grote problemen. Ze verbetert steepest descent door zoekrichtingen te kiezen die onderling A -conjugaat zijn, zodat eerdere vooruitgang niet ongedaan wordt gemaakt. Geometrisch betekent dit dat het minimum in opeenvolgende, optimaal gekozen richtingen wordt benaderd. Voor een exact kwadratische functie in n dimensies wordt het minimum in maximaal n stappen bereikt.

Globale minima. De besproken methodes zijn in essentie lokale optimalisatiemethodes en garanderen enkel convergentie naar een lokaal minimum. Het vinden van globale minima vereist aanvullende strategieën, zoals meerdere startpunten, globale zoekmethodes, randomisatie, of het combineren van lokale methodes met globale heuristieken. Zonder extra structurele informatie over de functie is het globale minimum in het algemeen niet efficiënt te bepalen.

- (a) Geef het Butcher-tableau van de methode van Heun.
- (b) Wat is een adaptive time step en waarom wordt die gebruikt?
- (c) Hoe zien expliciete en impliciete Butcher-tableaus er structureel uit?
- (d) Wat betekent FSAL en bij welke methodes komt dit voor?
- (e) Geef een concreet voorbeeld waarbij FSAL wordt gebruikt.

Antwoord.

- (a) De methode van Heun is een expliciete Runge–Kutta methode van orde twee. Ze combineert een Euler-voorspelling met een correctiestap op basis van het gemiddelde van twee hellingen. Het bijhorende Butcher-tableau is

$$\begin{array}{c|cc} 0 & & \\ 1 & 1 & \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array}$$

- (b) Een adaptive time step betekent dat de stapgrootte h tijdens de integratie van een differentiaalvergelijking automatisch wordt aangepast. Het doel is om een vooraf opgelegde fouttolerantie te respecteren met een minimale rekentijd. Dit gebeurt door de lokale truncatiefout per stap te schatten, typisch via twee methodes van verschillende orde, en op basis daarvan h te verkleinen of te vergroten.
- (c) Bij expliciete Runge–Kutta methodes is het Butcher-tableau strikt onderdriehoekig, wat betekent dat elke nieuwe helling enkel afhangt van reeds berekende hellingen. Hierdoor kan elke stap expliciet worden uitgevoerd. Bij impliciete methodes bevat het tableau niet-nul elementen op of boven de hoofddiagonaal, waardoor de hellingen onderling gekoppeld zijn en er per stap een (niet-lineair) stelsel moet worden opgelost. Impliciete methodes zijn duurder per stap maar hebben superieure stabiliteitseigenschappen.
- (d) FSAL staat voor *First Same As Last*. Dit principe houdt in dat de laatste functiewaarde die in een tijdstap wordt berekend, identiek is aan de eerste functiewaarde van de volgende tijdstap. Hierdoor kan één functiewaarde per stap worden uitgespaard, wat de efficiëntie verhoogt zonder de orde van de methode te verminderen.
- (e) Een bekend voorbeeld van een methode met het FSAL-principe is de Dormand–Prince Runge–Kutta methode van orde 5(4). Deze methode wordt veel gebruikt in combinatie met adaptive time stepping, omdat ze tegelijk een foutschatting levert en rekentijd bespaart door hergebruik van de laatste helling.

- (a) Leg uit hoe polynoomvermenigvuldiging kan worden uitgevoerd met behulp van FFT en convolutie.
- (b) Wat stelt de parameter n voor in FFT-gebaseerde polynoomvermenigvuldiging en waarom doen we aan zero padding?
- (c) Kunnen we ook andere waarden voor n kiezen dan de minimale vereiste? Wat is het effect daarvan?
- (d) Indien de coëfficiënten van de veeltermen reëel zijn, mogen we dan `fft` en `ifft` vervangen door `rfft` en `irfft`? Verklaar waarom wel of niet.

Antwoord.

- (a) Bij FFT-gebaseerde polynoomvermenigvuldiging wordt gebruikgemaakt van het feit dat vermenigvuldiging van veeltermen overeenkomt met convolutie van hun coëfficiënten. Gegeven twee veeltermen $p(x) = \sum_{k=0}^m a_k x^k$ en $q(x) = \sum_{k=0}^{\ell} b_k x^k$, zijn de coëfficiënten van het product $r(x) = p(x)q(x)$ gegeven door de discrete convolutie van de reeksen (a_k) en (b_k) . In plaats van deze convolutie rechtstreeks te berekenen in $O(n^2)$, worden beide coëfficiëntenreeksen via de FFT getransformeerd naar frequentieruimte, daar puntsgewijs vermenigvuldigd, en vervolgens via een inverse FFT teruggetransformeerd. Dit reduceert de rekentijd tot $O(n \log n)$.
- (b) De parameter n stelt de lengte voor van de discrete Fouriertransformatie die wordt gebruikt. Ze moet minstens zo groot zijn als het aantal coëfficiënten van het resultaat, dus $n \geq m + \ell + 1$. Zero padding wordt toegepast om de oorspronkelijke coëfficiëntenreeksen tot lengte n aan te vullen met nullen, zodat de circulaire convolutie die door de FFT wordt uitgevoerd overeenkomt met de gewenste lineaire convolutie van de veeltermen.
- (c) Andere waarden voor n dan de minimale vereiste zijn toegestaan, zolang n groot genoeg is om circulaire overlap te vermijden. In de praktijk wordt n vaak gekozen als een macht van twee, omdat FFT-algoritmen dan het efficiëntst zijn. Een grotere n verhoogt de rekentijd licht, maar kan de implementatie vereenvoudigen en soms numerieke afrondingsfouten beter spreiden.
- (d) Wanneer de coëfficiënten van beide veeltermen reëel zijn, mag men `fft` en `ifft` vervangen door respectievelijk `rfft` en `irfft`. In dat geval wordt gebruikgemaakt van de Hermitische symmetrie van de Fouriertransformatie van reële signalen, waardoor slechts de helft van het spectrum expliciet moet worden opgeslagen en berekend. Dit vermindert zowel het geheugengebruik als de rekentijd, zonder verlies aan informatie.

Examenversie 4 — Vraag 1 (Eigenwaarden en eigenvectoren)

- (a) Gegeven een benaderende eigenwaarde λ , hoe vind je de corresponderende eigenvector?
- (b) Gegeven een benaderende eigenvector v , hoe vind je de corresponderende eigenwaarde?
- (c) Gegeven een vierkante matrix A , hoe vind je:
 - (a) de grootste eigenwaarde van A ,
 - (b) de kleinste eigenwaarde van A ,
 - (c) de eigenwaarde van A die het dichtst bij een gegeven scalair b ligt,
 - (d) alle eigenwaarden van A ?
- (e) In een stuk code wordt power iteration gebruikt maar bevat deze enkele tekortkomingen. Welke typische verbeteringen of aandachtspunten zijn hier nodig?

Antwoord.

- (a) Gegeven een benaderende eigenwaarde λ , zoekt men een corresponderende eigenvector v door het homogene stelsel $(A - \lambda I)v = 0$ op te lossen. Omdat λ slechts benaderend is, wordt dit stelsel typisch niet exact singulier. In de praktijk zoekt men daarom een vector v die $(A - \lambda I)v$ minimaliseert, bijvoorbeeld via het oplossen van een least-squares probleem of door inverse iteration toe te passen met vaste λ .
- (b) Gegeven een benaderende eigenvector v , kan de corresponderende eigenwaarde worden geschat met de Rayleigh-quotient $\lambda = \frac{v^T A v}{v^T v}$. Deze keuze levert de beste scalaire benadering van A in de richting van v en is consistent met het eigenwaardeprobleem wanneer v exact een eigenvector is.
- (c)
 - (a) De grootste eigenwaarde van A kan worden gevonden met de power iteration. Deze methode herhaalt $v_{k+1} = Av_k / \|Av_k\|$ en convergeert naar de eigenvector die hoort bij de eigenwaarde met grootste absolute waarde, mits een geschikte startvector.
 - (b) De kleinste eigenwaarde kan worden gevonden door power iteration toe te passen op A^{-1} , wat neerkomt op inverse iteration. In de praktijk wordt dit gerealiseerd door per iteratie een lineair stelsel met matrix A op te lossen.
 - (c) De eigenwaarde die het dichtst bij een gegeven scalair b ligt, kan worden bepaald via shifted inverse iteration op $(A - bI)^{-1}$. Deze methode versterkt de bijdrage van de eigenwaarde die het dichtst bij b ligt en laat de iteratie daar naartoe convergeren.
 - (d) Alle eigenwaarden van A worden typisch bepaald met globale methodes zoals de QR-iteratie. Deze methode reduceert A eerst tot Hessenberg- of tridiagonale vorm en voert vervolgens herhaalde QR-decomposities uit totdat de matrix convergeert naar een (bijna) diagonale vorm met de eigenwaarden op de diagonaal.
- (e) Typische tekortkomingen in code voor power iteration zijn het ontbreken van normalisatie, waardoor overflow of underflow kan optreden, en het ontbreken van een goede stopcriterium gebaseerd op residu $\|Av - \lambda v\|$. Verbeteringen omvatten expliciete normalisatie per iteratie, het gebruik van de Rayleigh-quotient voor een betere eigenwaardeschatting, en een robuuste convergentietest in plaats van een vast aantal iteraties.

- (a) Leg uit wat numerieke kwadratuur is en bespreek de verschillen tussen Newton–Cotes, Clenshaw–Curtis en Gaussische kwadratuur.
- (b) Geef de drie laagste orde Newton–Cotes regels en maak een bijbehorende schets.
- (c) Ik wil een accurater resultaat dan de Simpsonregel mij geeft. Heeft het zin om dan een derdegraads polynoom te interpoleren?

Antwoord.

- (a) Numerieke kwadratuur heeft als doel de integraal $\int_a^b f(x) dx$ numeriek te benaderen door een gewogen som van functiewaarden, $\sum_{i=0}^n w_i f(x_i)$. Het verschil tussen kwadratuurmethodes zit in de keuze van de knooppunten x_i en de gewichten w_i .

Bij **Newton–Cotes** liggen de knooppunten gelijkmatig verdeeld over het interval en wordt f geïnterpoleerd door een polynoom. De integraal van dit interpolatiepolynoom levert de kwadratuurregel. Deze methode is eenvoudig maar kan bij hogere orde slecht geconditioneerd en oscillerend worden.

Clenshaw–Curtis gebruikt eveneens polynomiale interpolatie, maar met Chebyshev-knooppunten in plaats van equidistante punten. Dit beperkt oscillaties van het interpolatiepolynoom en leidt tot betere stabiliteit. De methode kan bovendien efficiënt worden geïmplementeerd met FFT.

Bij **Gaussische kwadratuur** worden zowel knooppunten als gewichten optimaal gekozen zodat de regel exact is voor polynomen van zo hoog mogelijke graad. Met n knooppunten is de methode exact tot graad $2n - 1$. Dit maakt Gaussische kwadratuur zeer efficiënt, maar de knooppunten zijn niet uniform en moeten vooraf worden berekend.

- (b) De drie laagste orde Newton–Cotes regels zijn:
 - de **rechthoekregel**, waarbij f wordt benaderd door een constante op het interval;
 - de **trapeziumregel**, waarbij f lineair wordt geïnterpoleerd tussen de eindpunten;
 - de **Simpsonregel**, waarbij f wordt geïnterpoleerd door een kwadratische veelterm die door de twee eindpunten en het midden van het interval gaat.

Een schets toont respectievelijk een rechthoek, een trapezium en een parabool die de grafiek van f op $[a, b]$ benaderen.

- (c) Het heeft in het algemeen geen zin om een derdegraads interpolerende veelterm te gebruiken om Simpson te verbeteren. Hoewel de interpolerende veelterm dan een hogere graad heeft, leidt een Newton–Cotes regel van hogere orde tot sterkere oscillaties tussen de knooppunten en kan de fout zelfs groter worden. Simpson is exact voor polynomen tot graad drie en vormt een stabiel compromis. Hogere nauwkeurigheid wordt beter bereikt door samengestelde Simpson, intervalopsplitsing, of door over te stappen op Clenshaw–Curtis of Gaussische kwadratuur in plaats van de polynomiale graad verder te verhogen.

- (a) Geef de *key ingredients* van een PRNG.
- (b) Beschrijf het algemene werkingsprincipe van een PRNG aan de hand van een flow chart (verbaal).
- (c) Geef enkele concrete, praktische voorbeelden van de key ingredients.
- (d) Is het mogelijk om twee keer hetzelfde getal te bekomen uit een PRNG? Verklaar.
- (e) Uit welke distributie samplet een PRNG?

Antwoord.

- (a) De kerncomponenten van een pseudo random number generator zijn:
 - een **interne toestand** (state) die de huidige status van de generator beschrijft;
 - een **deterministische update-regel** die de toestand bijwerkt;
 - een **output-functie** die uit de toestand een getal produceert;
 - een **initialisatie** via een startwaarde of *seed*;
 - een voldoende lange **periode**, zodat herhaling van de toestand pas na veel stappen optreedt.
- Samen bepalen deze elementen de statistische kwaliteit en reproduceerbaarheid van de gegenereerde getallen.
- (b) Het algemene werkingsprincipe van een PRNG kan als volgt worden beschreven. Men start met een initiële toestand die wordt vastgelegd door de seed. Deze toestand wordt via een vaste, deterministische update-regel omgezet in een nieuwe toestand. Uit deze nieuwe toestand wordt een getal gegenereerd via de output-functie. Dit proces wordt herhaald, waarbij elke nieuwe toestand zowel de volgende toestand als het volgende pseudo-toevalsgetal bepaalt. Ondanks het deterministische karakter vertoont de resulterende reeks statistisch willekeurig gedrag.
 - (c) Een concreet voorbeeld van een update-regel is een lineaire congruentie van de vorm $x_{n+1} = (ax_n + c) \bmod m$, waar a , c en m parameters zijn die de periode en kwaliteit bepalen. De interne toestand is hier het getal x_n , en de output wordt verkregen door x_n/m te nemen, wat een getal in $[0, 1)$ oplevert. Bij modernere generatoren bestaat de toestand uit meerdere gehele getallen en is de update-regel complexer, maar het principe blijft hetzelfde.
 - (d) Ja, het is mogelijk om exact hetzelfde getal meerdere keren te bekomen uit een PRNG. Omdat de generator een eindige interne toestand heeft en deterministisch is, zal de toestandsreeks zich na verloop van tijd herhalen. Bovendien kan dezelfde output ook voorkomen zonder dat de volledige toestand identiek is, afhankelijk van de output-functie. Dit onderscheidt pseudo random getallen fundamenteel van echte willekeur.
 - (e) Een PRNG samplet in zijn basisvorm uit een discrete uniforme verdeling over een eindige set van toestanden. Na normalisatie worden de gegenereerde waarden typisch geïnterpreteerd als realisaties uit een continue uniforme verdeling op het interval $[0, 1)$. Andere distributies worden niet rechtstreeks gesampled, maar verkregen door transformaties van deze uniforme pseudo random getallen.

- (a) Wat is een stiff ODE? Geef ook een voorbeeld.
- (b) Welke single-step methods hebben we gezien voor een stiff ODE op te lossen? Geef een voorbeeld en leg uit.
- (c) Welke multistep methods hebben we gezien voor stiff ODE's? Geef een voorbeeld en leg uit.

Antwoord.

- (a) Een ODE heet *stiff* wanneer de stabiliteitseisen van numerieke methodes een veel kleinere tijdstap vereisen dan wat nodig is voor nauwkeurigheid. Met andere woorden: de exacte oplossing kan traag en glad variëren, maar expliciete methodes worden instabiel tenzij een extreem kleine stapgrootte wordt gebruikt. Dit treedt typisch op wanneer het probleem meerdere tijdschalen bevat, bijvoorbeeld wanneer de Jacobiaan eigenwaarden heeft met sterk verschillende negatieve reële delen.

Een klassiek voorbeeld is het lineaire testprobleem $y'(t) = \lambda y(t)$ met $\lambda \ll 0$. Voor grote negatieve λ is de exacte oplossing snel uitdovend, maar expliciete methodes vereisen $h|\lambda| \ll 1$ voor stabiliteit, wat leidt tot zeer kleine stappen.

- (b) Voor stiff ODE's zijn impliciete single-step methodes geschikt omdat ze grote stabiliteitsgebieden hebben. Een standaardvoorbeeld is de **backward Euler methode**, gedefinieerd door $y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$. Deze methode is A-stabiel, wat betekent dat ze stabiel is voor alle stapgroottes voor het lineaire testprobleem met negatieve reële λ . Hierdoor kan men grote tijdstappen gebruiken zonder instabiliteit. Het nadeel is dat per stap een (niet-lineair) stelsel moet worden opgelost, wat de rekenkost verhoogt.

Andere impliciete single-step methodes die we gezien hebben zijn impliciete Runge–Kutta methodes, zoals de impliciete midpoint-methode, die eveneens A-stabiel zijn en hogere orde kunnen bereiken.

- (c) Voor stiff ODE's zijn expliciete multistep methodes ongeschikt wegens beperkte stabiliteitsgebieden. De belangrijkste multistep methodes voor stijve problemen zijn de **Backward Differentiation Formula (BDF)**. Deze methodes benaderen de afgeleide door een achterwaartse differentieformule en evalueren de rechterlidfunctie impliciet in t_{n+1} .

Een voorbeeld is BDF2: $\frac{3y_{n+1} - 4y_n + y_{n-1}}{2h} = f(t_{n+1}, y_{n+1})$. BDF-methodes van lage orde zijn A-stabiel of L-stabiel en laten daardoor grote stapgroottes toe bij stijve problemen. Ze zijn bijzonder efficiënt voor problemen waarbij de oplossing langzaam varieert maar sterke demping aanwezig is.

- (a) Stel een linear least-squares probleem $Ax \approx b$ met A een $m \times n$ matrix.
- (b) Wat als $\text{rank}(A) < n$? Welke situaties kunnen we krijgen?
- (c) Welke methodes hebben we gezien om dit probleem op te lossen (voor algemene A)? Wat zijn de voor- en nadelen van elke methode?

Antwoord.

- (a) Een linear least-squares probleem bestaat erin een vector $x \in \mathbb{R}^n$ te bepalen die de residunorm $\|Ax - b\|_2$ minimaliseert, waarbij $A \in \mathbb{R}^{m \times n}$ en typisch $m \geq n$. Geometrisch komt dit neer op het projecteren van b op de kolomruimte van A . De oplossing voldoet aan de orthogonaliteitsvoorwaarde $A^T(Ax - b) = 0$, wat leidt tot de normale vergelijkingen $A^T Ax = A^T b$ wanneer A volle kolomrang heeft.
- (b) Als $\text{rank}(A) < n$, dan is de kolomruimte van A lager-dimensionaal en is de nulruimte van A niet-triviaal. Dit heeft belangrijke gevolgen:
 - **Geen exacte oplossing:** in het algemeen ligt b niet in de kolomruimte van A , zodat $Ax = b$ geen oplossing heeft. Het least-squares probleem heeft wel oplossingen die het residu minimaliseren.
 - **Unieke oplossing:** deze situatie kan niet voorkomen wanneer $\text{rank}(A) < n$, omdat er dan altijd niet-triviale nulruimte bestaat.
 - **Oplossing, maar niet uniek:** dit is de typische situatie. Er bestaan oneindig veel x die hetzelfde minimale residu opleveren. Al deze oplossingen verschillen door elementen uit de nulruimte van A .

In dit geval wordt vaak de oplossing met minimale Euclidische norm geselecteerd.

- (c) Er bestaan verschillende numerieke methodes om een linear least-squares probleem op te lossen:
 - **Normale vergelijkingen:** men lost $A^T Ax = A^T b$ op. Deze methode is eenvoudig en snel, maar de conditionering van A wordt gekwadeerd, wat kan leiden tot grote numerieke fouten bij slecht geconditioneerde matrices.
 - **QR-decompositie:** men schrijft $A = QR$ met Q orthogonaal en R boventriangulair. Dit leidt tot een stabiel algoritme dan de normale vergelijkingen en vermijdt expliciete vorming van $A^T A$. De rekentijd is hoger, maar de numerieke betrouwbaarheid is aanzienlijk beter.
 - **Singular Value Decomposition (SVD):** men schrijft $A = U\Sigma V^T$. Deze methode is het meest robuust en behandelt rang-deficiëntie op een natuurlijke manier. Ze levert automatisch de minimale-norm oplossing, maar is ook de duurste methode computationeel.

De keuze van methode is een afweging tussen snelheid en numerieke stabiliteit, waarbij QR en SVD in de praktijk vaak verkozen worden boven de normale vergelijkingen.

- (a) Welke integraal probeert deze code uit te rekenen?

```
import numpy as np
print(np.mean(cosh(np.random.normal(0, 1, 10000))))
```

- (b) Hoe heet deze integratietechniek?

- (c) Moeten we ons hier zorgen maken over convergentie? Verklaar.

Antwoord.

- (a) De code

```
import numpy as np
print(np.mean(cosh(np.random.normal(0, 1, 10000))))
```

benadert de verwachtingswaarde van de functie $\cosh(x)$ wanneer x een standaardnormaal verdeelde toevalsvariabele is. Dit komt overeen met de integraal

$$\int_{-\infty}^{\infty} \cosh(x) \frac{1}{\sqrt{2\pi}} e^{-x^2/2} dx,$$

die geïnterpreteerd wordt als $E[\cosh(X)]$ met $X \sim \mathcal{N}(0, 1)$.

- (b) Deze techniek heet **Monte Carlo-integratie via sampling**. De integraal wordt herschreven als een verwachtingswaarde en vervolgens benaderd door het steekproefgemiddelde van onafhankelijke realisaties van de toevalsvariabele. De random getallen worden hier rechtstreeks gesampled uit de doelverdeling, namelijk de standaardnormale verdeling.
- (c) Ja, convergentie is hier een aandachtspunt. Monte Carlo-methodes convergeren weliswaar altijd volgens de wet van de grote aantallen, met een fout die daalt als $1/\sqrt{N}$, maar de efficiëntie hangt sterk af van de variantie van de integrand. De functie $\cosh(x)$ groeit exponentieel voor grote $|x|$, waardoor zeldzame maar grote bijdragen aan het gemiddelde kunnen voorkomen. Dit leidt tot een relatief grote variantie en dus trage statistische convergentie. Hoewel de verwachtingswaarde eindig is, zou men in de praktijk technieken zoals variance reduction of importance sampling overwegen om de efficiëntie te verbeteren.

Root finding

- (a) Waarom kan de Newton methode niet convergeren?
- (b) De secant methode gebruikt lineaire interpolatie. Waarom gebruiken we geen interpolatie van hogere graad?
- (c) Bij optimalisatie doen we wel iets gelijkaardigs. Welke methode is dit?
- (d) Leg twee methoden uit om alle nulpunten van een functie te vinden.

Antwoord.

- (a) De Newton methode kan niet convergeren omdat ze slechts lokaal convergent is en sterk afhankelijk is van de startwaarde. Als de beginwaarde te ver van het gewenste nulpunt ligt, kan de iteratie divergeren of naar een ander nulpunt convergeren. Een belangrijk faalmechanisme treedt op wanneer de afgeleide $f'(x)$ (bijna) nul is in een iteratiepunt, omdat de Newton-stap $-\frac{f(x)}{f'(x)}$ dan zeer groot wordt en de iteratie onstabiel maakt. Daarnaast kan Newton oscilleren of in een cyclus terechtkomen bij ongunstige functievormen, en bij nulpunten met hogere multipliciteit verliest de methode haar kwadratische convergentie en wordt ze slechts lineair convergent.
- (b) Interpolatie van hogere graad wordt in root finding meestal vermeden omdat ze numeriek fragieler is en geen robuuste winst garandeert. Hogere-graads interpolanten vereisen meer punten, verhogen de algebraïsche kost en zijn gevoeliger voor ruis en afrondingsfouten. Bovendien kunnen ze ongewenste oscillaties vertonen, waardoor het berekende nulpunt buiten een zinvol interval kan vallen. De secant methode vormt een goede balans: ze vermijdt afgeleiden, gebruikt slechts twee punten, en bereikt toch superlineaire convergentie zonder de instabiliteit die hogere-graads interpolatie kan introduceren.
- (c) In optimalisatie wordt een gelijkaardig idee gebruikt in methodes op basis van parabolische interpolatie. Hierbij wordt een kwadratische veelterm gefit door drie functiewaarden $(x_i, f(x_i))$, en wordt het minimum van deze parabool genomen als nieuw iteratiepunt. Deze aanpak vormt de kern van methodes zoals parabolic interpolation en wordt in de praktijk vaak gecombineerd met bracketing in robuuste algoritmen zoals Brent's methode voor minima.
- (d) Twee fundamentele strategieën om alle nulpunten van een functie te vinden zijn:
 - **Bracketing op een rooster gevolgd door lokale root finding:** men evalueert de functie op een fijn rooster, detecteert intervallen met tekenwissels, en past op elk interval een robuuste methode zoals bisection toe. Dit is betrouwbaar voor nulpunten met tekenwissel, maar kan nulpunten met even multipliciteit missen.
 - **Reductie tot een eigenwaardeprobleem (voor polynomen):** voor polynomen kan men alle nulpunten bepalen door de companion matrix te construeren en haar eigenwaarden te berekenen. Deze methode levert alle nulpunten tegelijk, inclusief complexe, maar is gevoelig voor conditionering bij hoge graad.

Fast Fourier Transform

- (a) Wat berekent de volgende code?
- (b) Wat doet de parameter n ?
- (c) Kunnen we n vervangen door een andere waarde? Zo ja, door wat en met welk effect?
- (d) Als a en b reëel zijn, moeten we dan `fft/ifft` vervangen door `rfft/irfft`?

```
def unnamed(a, b):  
    n = len(a) + len(b)  
    fft1 = fft(a, n)  
    fft2 = fft(b, n)  
    return ifft(fft1 * fft2)
```

Antwoord.

- (a) De code berekent de discrete convolutie van de reeksen a en b via de FFT. Wanneer a en b geïnterpreteerd worden als coëfficiënten van twee veeltermen, levert de output precies de coëfficiënten van het productpolynoom. De FFT zet beide reeksen om naar frequentieruimte, waar de convolutie wordt omgezet in een puntsgewijze vermenigvuldiging. De inverse FFT brengt dit resultaat terug naar de oorspronkelijke ruimte.
- (b) De parameter n bepaalt de lengte van de FFT. Door a en b met nullen aan te vullen tot lengte n , wordt voorkomen dat de circulaire convolutie die inherent is aan de FFT overlap veroorzaakt. Om de lineaire convolutie correct te bekomen, moet n minstens zo groot zijn als het aantal coëfficiënten van het resultaat, dus typisch $n \geq |a| + |b| - 1$.
- (c) Ja, n kan vervangen worden door elke waarde die groot genoeg is om circulaire overlap te vermijden, met name elke $n \geq |a| + |b| - 1$. In de praktijk kiest men n vaak als de kleinste FFT-vriendelijke lengte (bijvoorbeeld een macht van twee) die aan deze voorwaarde voldoet. Een grotere n verhoogt de rekentijd, maar kan de FFT-implementatie efficiënter maken afhankelijk van de gebruikte bibliotheek.
- (d) Als a en b reëel zijn, kan men `fft` en `ifft` vervangen door `rfft` en `irfft`. De Fouriertransformatie van een reëel signaal heeft Hermitische symmetrie, waardoor slechts de helft van het spectrum expliciet moet worden berekend en opgeslagen. Dit verlaagt zowel het geheugengebruik als de reken-tijd. Men moet er wel voor zorgen dat de inverse transformatie met `irfft` wordt uitgevoerd met de correcte lengte n , zodat het resultaat opnieuw reëel is en overeenkomt met de gewenste convolutie.